

KyberCrypt Web Service — Test Plan

Test plan for the account-based, post-quantum (ML-KEM) encrypted file-transfer web service in `web/`. Emphasis on the security-sensitive surfaces (auth, 2FA, sessions, API keys, CSRF, streaming crypto, audit log, billing webhook, backups).

1. Scope & objectives

In scope: the `web/` Go service and its templates/static assets, the backup kit (`web/deploy/backup/`), and the deploy config (`web/deploy/`).

Out of scope: the `relay/` component; Caddy/TLS termination; Stripe's own infrastructure (we test our webhook verification and checkout call, not Stripe).

Objectives:

1. Every security-review fix has a regression test that fails if the fix regresses.
2. Core user journeys (register → login → 2FA → send → receive → manage keys → change password → billing) work end to end.
3. The at-rest crypto round-trips correctly across file sizes, and tampering is detected.

2. Environments & setup

Item	Value
Build	<code>go build ./... in web/</code> (Go 1.24+; uses <code>crypto/mlkem</code> , <code>crypto/pbkdf2</code>)
Unit tests	<code>go test ./... in web/</code>
Vet	<code>go vet ./...</code>
Local run	<code>WEB_ADDR=127.0.0.1:8099</code> <code>WEB_BASE_URL=http://127.0.0.1:8099</code> <code>WEB_DATA_DIR=<tmp></code> <code>./web</code>
Audit key	set <code>AUDIT_HMAC_KEY</code> for keyed-HMAC tamper-evidence tests
Backup tests	require <code>gpg</code> on <code>PATH</code>
Browser tests	a real browser (CSRF/Referrer-Policy interaction can't be fully covered by <code>curl</code>)

Note on local HTTP: cookies are Secure only when `WEB_BASE_URL` is `https://`, so cookie-flag assertions (TC-SEC-09) must run against an HTTPS deployment.

3. Test types & current automated coverage

Type	Tooling	Status
Unit — streaming crypto	<code>stream_test.go</code>	■ roundtrip (incl. chunk boundaries), wrong-key, tamper, truncation, bad-version, filename pipeline
Unit — audit chain	<code>audit_test.go</code>	■ clean verify, tamper, tail-truncation, restart continuity
Integration (HTTP)	none yet	■ gap — see §8

Type	Tooling	Status
Manual functional	this plan \$5	manual
Security regression	this plan \$6	partly manual; should be automated (\$8)

4. Core feature areas

Accounts/auth · Sessions · 2FA (TOTP + recovery) · Send-by-username (ML-KEM + streamed AEAD) · Inbox (list/download/delete) · API keys (send/decrypt scopes, expiry) + Bearer API · Password change · Billing (checkout + webhook) · Audit log · Backups · Security headers/CSP · Rate limiting.

5. Functional test cases (manual / integration)

Pre-req for most: register two users (alice, bob), include Origin header on state-changing requests, and carry the session cookie + CSRF token (from the `<meta name="csrf-token">` on /app).

ID	Area	Steps	Expected
TC-F-01	Register	POST /api/register valid username + 12+ char password	200 {ok:true}, session cookie set, account file created
TC-F-02	Register	username not matching <code>^[a-z0-9_-]{3,32}\$</code>	400
TC-F-03	Register	password < 12 chars	400
TC-F-04	Register	duplicate username	400 "username already taken"
TC-F-05	Login	correct creds, no 2FA	200 {ok:true}, session cookie
TC-F-06	Login	wrong password	401
TC-F-07	Send	alice→bob, small file	200; message appears in bob's inbox
TC-F-08	Send	unknown recipient	404 "no such recipient"
TC-F-09	Inbox	bob lists inbox (UI + /api/v1/inbox)	message listed with sender/size
TC-F-10	Download	bob downloads → decrypts	bytes + filename match original (sha256)
TC-F-11	Delete	bob deletes message	gone from inbox + disk
TC-F-12	API key	create decrypt key; use Bearer to send + download	works; roundtrip OK
TC-F-13	API key	create send key; attempt /api/v1/inbox	403 "send-only"
TC-F-14	API key	expired key (set expires_days, age it)	401 "expired"
TC-F-15	API key	revoke key; reuse	401
TC-F-16	2FA	setup → enable with valid TOTP	200, 10 recovery codes returned
TC-F-17	2FA	login → TOTP second step	200 after valid code
TC-F-18	2FA	login with recovery code	200; that code single-use afterwards
TC-F-19	2FA	disable with valid code	200; TOTP fields + recovery codes cleared
TC-F-20	Password	change with correct current password	200; private key re-wrapped, can still log in with new pw
TC-F-21	Billing	webhook checkout.session.completed (valid sig)	plan → pro, audit plan_upgraded

ID	Area	Steps	Expected
TC-F-22	Billing	webhook customer.subscription.deleted	plan → free
TC-F-23	Plans	Pro user sends file > free cap	accepted up to Pro's MaxFileBytes

6. Security regression matrix (one row per review fix)

ID	Threat / fix	Test	Expected
TC-SEC-01	XFF spoofing → lockout bypass / forged IP	send X-Forwarded-For : 1.2.3.4 (spoof) behind proxy; check audit IP + lockout key	uses rightmost (proxy-appended) IP, not the spoofed left value
TC-SEC-02	Password change must revoke creds	after change: reuse old API key; reuse old session cookie	both rejected (401 / 303-to-login); new cookie works
TC-SEC-03	Recovery-code double-spend (race)	submit same recovery code in two concurrent logins	at most one succeeds
TC-SEC-04	Upload memory bound	send file ■ multipart threshold; observe RSS	bounded (streams to disk/blob, not whole-file in RAM)
TC-SEC-05	Username enumeration timing	time login for existing vs nonexistent user	comparable latency (KDF burned on miss)
TC-SEC-06	Authenticated backups	tamper a .gpg backup, restore	restore fails (integrity)
TC-SEC-07	TOTP replay	reuse same TOTP code at a second login (same window)	401; but enrollment-window login still works (TC-F-17)
TC-SEC-08	CSRF — origin	state-changing POST with foreign/absent Origin	403
TC-SEC-09	CSRF — token	authed POST missing/wrong X-CSRF-Token (or csrf_token)	403; correct token → 200
TC-SEC-10	CSRF — exemptions	Bearer /api/v1/* and /api/billing/webhook without token	allowed (not cookie-authed)
TC-SEC-11	Registration rate-limit	>20 registrations/hr from one IP	429
TC-SEC-12	2FA setup rate-limit	>20 setups/hr for one user	429
TC-SEC-13	Login lockout	6 failed logins (user or IP)	429 after 5
TC-SEC-14	Per-plan upload cap	free user sends > 25 MiB	413 "exceeds your plan's size limit"
TC-SEC-15	Streamed blob integrity	flip a byte in a stored .blob; download	decryption fails (not silent corruption)
TC-SEC-16	Streamed blob truncation	drop the final chunk of a .blob; download	fails (authenticated final-chunk flag)
TC-SEC-17	Audit tamper	edit an audit.log entry; restart / verify()	"hash mismatch at seq N"
TC-SEC-18	Audit truncation	drop last log line, keep audit.head	"head mismatch (log truncated?)"
TC-SEC-19	Webhook signature	webhook with bad/absent Stripe-Signature or stale timestamp	400, no plan change
TC-SEC-20	Auth required	hit each /api/* (non-public) without session/Bearer	401
TC-SEC-21	Path traversal	id not matching validID; username with / or . .	rejected (regex + filepath.Base)
TC-SEC-22	Breached password	register/change with a known-pwned password	400 (fail-open if HIBP unreachable)

7. Non-functional

ID	Area	Test	Expected
TC-NF-01	Large transfer	round-trip a file near the plan cap	sha256 matches; memory bounded

ID	Area	Test	Expected
TC-NF-02	Concurrency	WEB_MAX_UPLOADS simultaneous + 1 uploads	excess queues; no OOM
TC-NF-03	Headers/CSP	inspect response headers	HSTS, nosniff, X-Frame-Options: DENY, strict CSP, Referrer-Policy: no-referrer
TC-NF-04	Restart durability	restart server mid-life	sessions cleared (in-memory by design); accounts/inbox/audit persist
TC-NF-05	Expiry sweep	message past TTL	dropped by sweep / lazily on list

8. Gaps & recommended automation

- **No HTTP-level integration tests.** The §5/§6 cases are currently manual

(we ran them as ad-hoc curl smoke tests). Recommend a `*_test.go` suite using `net/http/httptest.Server` over a temp `WEB_DATA_DIR`, asserting the matrix above — especially TC-SEC-02/07/08/09/15/16 and the auth-required sweep.
- **Browser-level CSRF check** (TC-SEC-08/09) needs a real browser to confirm the Origin header is sent despite Referrer-Policy: no-referrer; cover with one Puppeteer/Playwright test.
- **Timing test** (TC-SEC-05) is inherently flaky; treat as a coarse manual check.

9. Exit criteria

- `go build, go vet, go test ./...` all green.
- All TC-SEC-* pass (these are the regression guarantees for the hardening work).
- Core journeys TC-F-01..23 pass on a staging deploy with `AUDIT_HMAC_KEY` set and `WEB_BASE_URL` on HTTPS.
- Backup → restore verified on the target OS with `gpg` installed.